

프로젝트 보고서

TCP 기반 원격 임베디드 장치 제어 데몬 시스템
작성자: 전제혁

1. 프로젝트 개요

1-1. 개발 배경 및 목적

본 프로젝트는 호스트 PC(Ubuntu Linux)와 원격지에 위치한 타겟 임베디드 장비(Raspberry Pi 4) 간의 TCP/IP 소켓 통신을 구축하고, 주변 하드웨어 소자들을 안정적으로 제어 및 모니터링할 수 있는 완전 비동기식 관제 시스템을 개발하는 데 목적이 있습니다. 특히, 임베디드 제어 시스템의 핵심 요구조건인 무중단 상시 운영을 충족하기 위해 서버 본체를 백그라운드 상주형 데몬 프로세스로 전환하였으며, 런타임 의존성을 완전히 분리하여 시스템 유연성을 확보하고자 하드웨어 제어 로직을 동적 공유 라이브러리 형태로 분할 컴파일하는 정석 임베디드 아키텍처를 구현하였습니다.

1-2. 시스템 구성 및 개발 환경

- 서버 (**Target Board**): Raspberry Pi 4 (Raspberry Pi OS v8), 백그라운드 무중단 데몬 모드, 커널 로깅 시스템(syslog) 통합
- 클라이언트 (**Host PC**): Ubuntu Linux 기반 호스트 환경, `select()` 다중 I/O 멀티플렉싱 CLI, 예기치 못한 종료 방지를 위한 시그널 마스킹 가드 구현
- 제어 하드웨어 (**Target Devices**):
 - LED: 소프트웨어 PWM 기법을 적용한 원격 3단계 밝기 조절(HIGH/MID/LOW) 및 온오프 제어
 - 조도 센서 (CDS): I2C 버스 통신 인프라 기반 실시간 아날로그 조도 수치 수집 및 자동 조명 점등 제어
 - 7-세그먼트: 타이머 연동형 카운트다운 숫자 디스플레이 및 0초 도달 시 물리 인터럽트 이벤트 핸들링
 - 부저 (Buzzer): `softTone` 주파수 제어를 이용한 스크립트 기반 배경음악(학교종) 연주 및 `volatile` 동시성 제어 종료 플래그 적용
- 빌드 시스템: 크로스 컴파일 아키텍처(`aarch64-linux-gnu-gcc`) 및 모듈화된 `Makefile` 통합 컴파일 가동, 원클릭 배포 쉘 스크립트(`scp.sh`) 설계

2. 시스템 아키텍처 및 모듈 설계

2-1. 소프트웨어 계층 구조 및 분리 설계

프로젝트의 독립적 형상 관리 및 빌드 오버헤드 감소를 위해, 소스코드를 역할과 동작 인프라에 따라 3개의 디렉토리 구조(`client`, `server`, `device`)로 완전히 분리했습니다. 최종 컴파일 산출물은 호스트 PC 최상위 루트 디렉토리의 `.bin/` 폴더로 집결되며, 전송용 스크립트에 의해 타겟 보드로 이행됩니다.

- **client/** : 사용자의 키보드 입력 및 소켓 수신 이벤트를 비동기로 추적하는 독자적 콘솔 인터페이스부
- **server/** : 다중 커백션 처리를 위한 **pthread** 멀티스레딩 네트워크 제어부 및 **Stevens** 규격을 준수하는 시스템 커널 데몬 코어
- **device/** : wiringPi 및 하드웨어 물리 레지스터 제어 로직이 캡슐화된 순수 동적 라이브러리 소스 세트

2-2. 동적 라이브러리(Dynamic Loading) 데이터 파이프라인

메인 서버 코어는 컴파일 타임에 물리 하드웨어 종속성(wiringPi 라이브러리 등)을 일절 갖지 않습니다. 클라이언트로부터 제어 패킷이 도달하면 명령어 분기 처리부(parse_and_execute)가 동작하여 해당 장치 제어 전달 pthread 스레드를 생성합니다. 워커 스레드는 독립적인 가상 메모리 공간 내부에서 dlopen() 인터페이스를 호출하여 런타임에 libdevice.so 핸들을 획득하고, dlsym()으로 각 제어 함수의 진입점 주소를 바인딩하여 제어 플로우를 실행한 뒤 dlclose()로 안전하게 메모리를 반환하는 플러그인 아키텍처로 구동됩니다.

3. 세부 구현 내용

3-1. 백그라운드 상주형 데몬 서버 코어

서버의 진입점 메인 루프 가동 시 become_daemon() 헬퍼 함수가 기동됩니다. umask(0) 설정을 통해 파일 생성 권한을 완전히 개방하고, fork()를 단행한 뒤 제어 터미널을 보유한 부모 프로세스를 터미널 세션에 즉시 반환(exit)시킵니다. 이후 자식 프로세스는 setsid()를 통해 독자적인 새로운 프로세스 그룹 세션 리더가 되어 터미널 제어권을 완전히 이탈합니다. 시스템 자원 누수 및 오작동 차단을 위해 상속받은 모든 파일 디스크립터를 깨끗하게 클로즈(close)한 후 표준 입출력 장치(0, 1, 2번 디스크립터)를 커널 쓰레기통 장치인 /dev/null로 안전하게 리다이렉트 처리했습니다. 화면 출력이 불가한 특성은 커널 로그 시스템인 openlog() 및 syslog(LOG_DAEMON)규격으로 전면 변환하여 라즈베리파이 백그라운드 감시 체계를 구현했습니다.

3-2. select() 다중 I/O 멀티플렉싱 클라이언트

클라이언트 메인 루프 내부에서는 fd_set 구조체를 활용한 I/O 멀티플렉싱 기법을 적용했습니다. 표준 입력 장치인 키보드(0번)와 서버 연결용 TCP 소켓 디스크립터를 동시 감시 세트로 지정하고 select() 시스템 콜을 통해 커널 이벤트 처리를 블로킹 모드로 대기시킵니다. 사용자가 키보드 메뉴 입력을 발생시키면 0번 비트가 점등되어 패킷 조립 및 송신 루틴이 트리거되고, 반대로 사용자의 입력이 없더라도 라즈베리파이 센서 모듈로부터 측정 패킷이 날아오면 소켓 디스크립터 비트가 점등되어 화면에 동기화되는 논블로킹(Non-blocking) 비동기 사용자 CLI 환경을 구현했습니다. 프로그램 운용 안정성을 극대화하고자 sigprocmask()를 구동하여 SIGINT(Ctrl+C) 단일 예외 종료 신호를 제외한 전역 시스템 시그널을 블로킹 가드로 마스킹했습니다.

3-3. 하드웨어 주변장치 제어 모듈

각 제어 소스들은 동적 공유 객체 내부에 매립되는 특성을 지니므로 내부 동기화 및 전역 상태 관리가 중요합니다. setup.c 내에 static int is_global_init 래핑 플래그를 두어 멀티스레드 환경에서 하드웨어 물리 베이스(wiringPiSetup 등)가 수차례 중복 초기화되어 오작동하는 버그를 소프트웨어 방어 로직으로 차단했습니다. 부저 재생 모듈인 buz.c에서는 악보 데이터를 순회 연주하는 도중 클라이언트가 강제 정지 명령(OFF)을 전달하면, 컴파일러 최적화 연산을 배제하고 매번 메모리 물리 레지스터 값을 직참조하도록 강제하는 volatile int stop_flag 최적화 플래그 변수를 통해 재생 스레드를 실시간으로 안전하게 강제 통제하는 스레드 동시성 구조를 구현했습니다.

4. 문제점 및 보완 사항 (트러블 슈팅)

4-1. `select()` 블로킹 진입 전 출력 스트림 라인 버퍼링 현상

- 문제 현상: 클라이언트 프로그램 실행 시 사용자 메뉴 프롬프트 화면('input[1:LED]...: ')이 콘솔창에 즉각 인쇄되지 않고, 키보드로 사용자가 아무 값이나 치고 엔터를 눌러 이벤트를 강제 발생시켜야만 한 박자 밀려서 프롬프트 문자열이 뒤늦게 쏟아지는 치명적 화면 락 현상 확인.
- 원인 분석: 리눅스 시스템 가상 파일 파일 시스템의 표준 출력 장치('stdout')는 스트림 문자열 끝에 개행 문자('\n')가 도달하거나 내부 캐시 버퍼가 가득 차야만 하드웨어 모니터로 방출하는 '라인 버퍼링(Line Buffering)' 속성을 취함. 메뉴판 특성상 동일 라인에 입력을 받기 위해 끝에 '\n'을 배제하자 버퍼 스트림이 출당되었고, 프로세스가 곧바로 다음 줄인 '`select()`' 시스템 콜의 커널 블로킹 내부로 침전되어 들어가 버려 발생한 현상임.
- 해결 방법: 문자열을 출력한 후 '`select()`' 대기 구문으로 제어권이 넘어가기 직전, 메모리에 머물러 있는 스트림 장치를 물리적 모니터로 즉시 방출하도록 강제 강제하는 `fflush(stdout)`; 인터페이스 코드를 매립하여 즉각적인 UI 반응성 복구 완료.

4-2. 분할 동적 바인딩 컴파일 환경에서의 링커(Linker) 참조 충돌 에러

- 문제 현상: 전체 프로젝트의 빌드 자동화를 구현하기 위해 통합 마스터 **Makefile** 규칙을 수립하고 빌드 테스트를 단행하자, 메인 서버 본체 소스('server.c') 빌드 타임에 하드웨어 공통 핀 환경 설정을 수행하는 `device_init()` 선언부에서 **undefined reference to 'device_init'** 참조 충돌 링킹 에러를 터트리며 빌드 체인이 실패함.
- 원인 분석: 아키텍처 설계 단계에서 서버 본체는 순수 네트워크 관제 기능만 수행하도록 하드웨어 드라이버 링크 의존성 플러그('-ldevice / -lwiringPi')를 **Makefile** 빌드 옵션에서 원천 차단해 둠. 그러나 '`server.c`' 정적 메인 함수 코드 구문 상단에서 초기화 함수를 일반 C언어 함수 호출 형태로 직컴파일 참조를 시도함에 따라 컴파일러(링커)가 빌드 타임에 해당 가상 메모리 매핑 구역을 발견해 내지 못해 발생한 충돌임.
- 해결 방법: 하드웨어 초기화 구문인 '`device_init()`' 역시 정석적인 동적 바인딩 아키텍처 구조 규격으로 통합 리팩터링을 단행함. 메인 시작 타임에 런타임으로 '`dlopen()`'을 실행해 라이브러리 핸들을 따내고, '`dlsym()`'으로 가상 주소를 탐색한 뒤 함수 포인터 타입으로 간접 참조 실행하는 구조로 전환하여 컴파일 타임 결함도를 완전히 제거하고 빌드 성공 처리함.

4-3. 데몬 프로세스 `chdir("/")` 호출에 따른 공유 라이브러리 상대 경로 탐색 유실 문제

- 문제 현상: 서버 코어를 백그라운드 데몬화 시키는 루틴을 완비한 후 가동하자, 무선 패킷 수신은 완벽히 동작하나 클라이언트 제어 신호가 인입될 때 내부 워커 스레드단에서 실행되는 '`dlopen("/libdevice.so", ...)`' 구문이 전부 라이브러리를 발견하지 못하고 '`dlopen 실패`' 하드웨어 오작동 커널 에러 로그를 쏟아내며 먹통이 됨.
- 원인 분석: Stevens의 유닉스 정석 데몬 프로세스 표준 전환 로직 내부 규격에 명시된 시스템 보호 옵션에 의해, 세션 분리 직후 `chdir("/")` 구문이 수행됨. 이에 따라 가동 중인 백그라운드 프로세스의 가상 런타임 현재 작업 디렉토리(Current Working Directory)가 최초 실행 폴더 위치가 아닌 리눅스 최상위 시스템 루트인 '/' 폴더로 강제 락인 이탈하게 됨. 결과적으로 스레드들이 내부에 지정했던 라이브러리 탐색 상대 경로인 '`./libdevice.so`' 기호가 영동한 시스템 전역 최상위 루트 밑의 '`/libdevice.so`'를 타깃하게 되면서 로딩 인프라가 실패함.

- 해결 방법 (발상의 전환 - 완벽한 임베디드 최적화 **KISS 원칙**): 절대 경로 문자열 하드코딩이나 쉘 배포 시 무거운 전역 시스템 보관 폴더인 `'usr/lib'` 전역 이주 및 권한 제어 돌파용 `'sudo ldconfig'` 배포 보관 스크립트 구축 등의 대안을 설계해 봄. 그러나 이 우회로들은 환경 이식성을 해치고 스크립트 오버헤드가 막대해지는 딜레마가 수반됨.
 이에 프로세스의 시스템적 구동 원리를 원천 역추적하여 **"임베디드 단독 구동 가동 인프라 환경에서 `chdir("/")` 마운트 보호 보드가 실질적으로 필수적인가?"**에 대한 아키텍처 레벨의 근본적 의문을 제기함. 정형 인프라 서버 환경에서 루트 경로로 프로세스 위치를 강제 이탈시키는 핵심 사유는 임시 마운트 볼륨(**USB**, 외장 디스크 등)에서 실행된 데몬이 해당 폴더 디렉토리를 물리적으로 옮겨주고 있어 발생하는 시스템 볼륨 언마운트 불능 상태 방어 장치임. 그러나 본 라즈베리파이 원격 제어 타겟 시스템은 고정 온보드 플래시 메모리(**SD카드** 고정 특정 계정 내 홈 디렉토리) 내부에 박제 배포되어 독립 상시 상주 구동되는 인프라 구조이므로, 마운트 해제 지연이라는 리스크 시나리오 자체가 성립하지 않음을 명쾌하게 간파해 냄.
 결과적으로 데몬 프로세스 내부 가드 중 `'chdir("/")` 소스코드 단 한 줄을 완벽하게 삭제 차단함. 이를 통해 무거운 전역 시스템 디렉토리 매립 스크립트나 지저분한 절대 경로 문자열 결합 코드 증식 없이도, 데몬 프로세스가 최초 실행되었던 전용 프로젝트 폴더 주소를 **100%** 안전하게 유지하게 만들었으며, 상대 경로로 매명된 `'./libdevice.so'` 공유 객체를 칼같이 완벽 추적해 내는 임베디드 환경 최적화 를 구현함.

5. 일별 진척 상황

프로젝트 수행 기간(2026-06-02 ~ 2026-06-05) 동안의 단계별 상세 진척 결과물 및 개발 활동을 정리한 세부 진척 내역 현황표입니다.

일차	개발 단계 및 과제	세부 개발 활동 내용 및 산출물
1일차	아키텍처 설계 & 하드웨어 모듈화	- 프로젝트 폴더 분리 격리 및 서랍장식 마스터 빌드 시스템(Makefile) 구조 확립 - 라즈베리파이 하드웨어 독립 모듈화 설계 및 동적 라이브러리('libdevice.so') 추출 테스트 - 우분투 로컬 크로스 빌드 및 원격 타겟 보드 무선 이행 자동화 쉘 스크립트('scp.sh') 엔진 구축
2일차	비동기 멀티스레딩 & 다중 I/O 제어 이식	- 정적 컴파일 결합을 완전 배제하고 워커 'pthread' 내 런타임 'dlopen' 라이브러리 이식 완성 - 우분투 클라이언트 단 'select()' 엔진 탑재, 키보드 입력 및 비동기 소켓 데이터 수신 통로 개설 - 주파수 멜로디 재생 제어권 통제를 위한 'volatile' 종료 가드 플래그 상호 스레드 제어 메커니즘 검증 성공

일차	개발 단계 및 과제	세부 개발 활동 내용 및 산출물
3일차	데몬 프로세스 & 트러블 슈팅	- 백그라운드 완전 자립 무중단 동작 상주형 유닉스 표준 시스템 데몬 코어('fork', 'setsid') 이식 완성 - 'printf' 소멸 대응을 위한 커널 로깅 보관소 연동 체계('syslog') 리팩터링 및 실시간 감시 파이프라인 완비 - 데몬 런타임 경로 실종 버그 상황 발생, 구조 최적화 역발상('chdir' 탈락 제거)에 의한 통합 연동 패스 구현

* 본 보고서 문서의 설계 규격은 한화비전 VEDA 심화 실습 평가 평가 가이드라인을 준수합니다.